

1. Instructions Form a Stack

An agent receives instructions from more than the user message. The active stack may include system policy, project configuration, global and repository AGENTS.md files, nested guidance, skills, plugin instructions, tool contracts, and the current prompt. The practical question is not only whether each instruction is good, but whether the set is coherent.

OpenAI specifically recommends auditing skills and files such as AGENTS.md for Astra because stronger instruction following makes hidden or stale guidance more consequential. A rule that an older model often ignored can become an active constraint.

2. Scope and Precedence

AGENTS.md guidance is hierarchical. A repository-level file can establish broad conventions, while a deeper file can specify rules for its subtree. More specific project guidance should not be used to smuggle a global preference into one folder. Direct current instructions remain the strongest statement of what the user wants for the present task, subject to higher-level safety and tool constraints.

The important design property is locality: put a rule where its intended scope can be inferred from its location. A command required for the whole monorepo belongs high. A mobile-only snapshot workflow belongs near the mobile package.

3. Authorization Is an Instruction Too

A capable agent needs both a goal and a boundary. "Prepare the release" does not automatically mean publish it. "Draft the email" does not mean send it. "Investigate the billing error" does not mean purchase or cancel a plan.

Separate epistemic freedom from consequential authority. The agent may be allowed to research, inspect, run tests, draft, and stage changes broadly while still being required to ask before sending, publishing, purchasing, deleting data, changing account settings, or performing an irreversible operation.

4. Contradiction

Suppose AGENTS.md says "always run the full integration suite after any code change" while the current task says "do not run paid integration tests." The conflict is not solved by making both sentences longer. The current explicit boundary should control, and the durable rule should be revised if it routinely collides with legitimate tasks.

A useful audit question for every durable instruction is: Can I state a realistic case in which this rule should not apply? If yes, either encode the exception, narrow the scope, or move the instruction into a task-specific workflow.

5. Hidden Instruction Failure

The most difficult failures are often not visible in the user's prompt. A skill may require redundant screenshots. A legacy testing guide may force an expensive suite after every tiny edit. An old design instruction may ban a component now used everywhere else. Astra can execute these faithfully and thereby appear stubborn.

Before escalating reasoning because the agent "won't listen," inspect the instruction surfaces it can see. Stronger obedience to contradictory context is not repaired by louder repetition.

6. A Minimal Rule Test

Keep a durable rule only if all four conditions hold:

1. recurrence: it applies across many tasks in the scope;
2. stability: it is unlikely to change week to week;
3. actionability: the agent can tell what to do differently because of it;
4. non-redundancy: a linter, type checker, test, or existing tool does not already enforce it more reliably.

If a mechanical constraint can be enforced, prefer enforcement to prose. The best instruction file is not the one that mentions every failure ever seen. It is the one that changes behavior at the right moments.

References: OpenAI, Codex AGENTS.md documentation; OpenAI, Codex customization; OpenAI, GPT-6 Astra model guidance; Jason Liu, "Three Ways Codex Can Use a Computer."

Independent Open Course

<https://github.com/dicnunz/codex-astra-course>

CODEX.00 | Codex with GPT-6 Astra

Handout 5: Instruction Hierarchy

Fall 2026

License and source

Structural and pedagogical adaptation of MIT OpenCourseWare 24.00 Problems of Philosophy, Fall 2019, Prof. Miriam Schoenfield.

Original course and this adaptation are licensed CC BY-NC-SA 4.0.

This course is independently produced and is not affiliated with or endorsed by MIT or OpenAI.

Linked external readings and referenced product documentation retain their own copyrights and terms.

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

<https://ocw.mit.edu/courses/24-00-problems-of-philosophy-fall-2019/>